Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LESSON PLAN

For

Build ASP.NET Web Apps with Vibe Coding

TGS Ref No: TGS-2023040477

Software Design · ICT-DES-3005-1.1

Conducted by

Tertiary Infotech Academy Pte Ltd

Version 6.1 · 12 September 2026

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
6.1	12 September 2026	Two-day/16-hour delivery plan aligned to the v6.1 deck, ten cumulative LocalCRM Activities and the five official LO/K/A pairs.	Dr. Alfred Ang

TABLE OF CONTENTS

Course Overview

Learning Outcomes

Daily Schedule

Topic by Topic Breakdown

Resources Required

Assessment

Course Overview

Build ASP.NET Web Apps with Vibe Coding is a 2 days / 16 hours WSQ course aligned to Software Design (ICT-DES-3005-1.1). Learners build one cumulative local ASP.NET Core 10 CRM across ten self-contained Activity snapshots, progressing from syntax and component design through accessible Razor Pages, SQLite, controller APIs, interoperability, production readiness and design documentation.

Learning Outcomes

- LO1: Determine basic software components using ASP.NET framework to meet functional specifications.
- LO2: Utilize ASP.NET framework to develop web apps in adherence to organizational practices.
- LO3: Select essential ASP.NET controls and features to meet software design requirements.
- LO4: Examine the interoperability and functionality of ASP.NET software components.
- LO5: Generate ASP.NET design documentation aligned with user specifications.

Daily Schedule

Day 1 · 09:00–18:00

Time	Duration	Topic / Activity	Method	Slides
09:00–09:30	30 min	Administration, attendance, introductions and assessment briefing	Facilitation	—
09:30–10:45	75 min	Topic 1 — ASP.NET Core runtime and component design	Lecture, demonstration, questioning	Slides 18–58
10:45–10:55	10 min	Morning break	Break	—
10:55–12:30	95 min	Activities 1–2 — requirements, scaffold and controlled vibe change	Practical	Slides 52–57
12:30–13:30	60 min	Lunch	Break	—
13:30–14:50	80 min	Topic 2 — accessible UI and organisational practice	Lecture, demonstration, questioning	Slides 59–99
14:50–15:40	50 min	Activities 3–4 — accessible shell and audit	Practical	Slides 93–98
15:40–15:50	10 min	Afternoon break	Break	—
15:50–17:00	70 min	Topic 3 — Razor Pages controls, binding and persistence	Lecture and demonstration	Slides 100–140
17:00–17:50	50 min	Activity 5 — create and list with Razor Pages	Practical	Slides 134–136
17:50–18:00	10 min	Day 1 recap and PM attendance	Reflection	—

Day 2 · 09:00–18:00

Time	Duration	Topic / Activity	Method	Slides
09:00–09:15	15 min	Day 1 recap and AM attendance	Review	—
09:15–10:05	50 min	Topic 3 — validation, Tag Helpers and antiforgery	Lecture and demonstration	Slides 100–140
10:05–10:55	50 min	Activity 6 — validation and antiforgery	Practical	Slides 137–139
10:55–11:05	10 min	Morning break	Break	—
11:05–12:10	65 min	Topic 4 — controller APIs and HTTP contracts	Lecture, demonstration, questioning	Slides 141–181
12:10–13:00	50 min	Activity 7 — controller API	Practical	Slides 175–177

Time	Duration	Topic / Activity	Method	Slides
13:00–14:00	60 min	Lunch	Break	—
14:00–14:55	55 min	Activity 8 — SQLite, OpenAPI and integration tests	Practical	Slides 178–180
14:55–15:35	40 min	Topic 5 — hosting, Azure and design evidence	Lecture and demonstration	Slides 182–222
15:35–15:45	10 min	Afternoon break	Break	—
15:45–16:00	15 min	Activities 9–10 briefing and course synthesis	Demonstration and reflection	Slides 216–221
16:00–17:00	60 min	Written Assessment — Short-Answer Questions (open book)	Summative assessment	—
17:00–18:00	60 min	Practical Performance Assessment (open book)	Summative assessment	—

Topic-by-Topic Breakdown

Topic 1: Design Components and the ASP.NET Core Runtime · Slides 18–58

Alignment: LO1 · K1 / A1. requirements → component boundaries → request pipeline → verified scaffold.

- HTTP request and response contract
- ASP.NET technology family boundary
- SDK, runtime and target framework
- Project template and file ownership
- Composition root and dependency injection
- Configuration provider precedence
- Middleware pipeline ordering
- Vibe coding control loop

Activity 1: Translate Requirements into a Verified ASP.NET Core Scaffold · Slides 52–54

Turn a LocalCRM customer-contact brief into component boundaries and a reproducible .NET 10 Razor Pages project. Evidence: requirements.md, component-map.md, pinned SDK file and a clean net10.0 Razor Pages scaffold.

Activity 2: Run a Controlled Vibe-Coding Change · Slides 55–57

Add a health endpoint through a specify–generate–inspect–verify loop without accepting unrelated AI edits. Evidence: a reviewed /health endpoint, build evidence and a decision log.

Topic 2: Accessible UI and Organisational Vibe-Coding Practice · Slides 59–99

Alignment: LO2 · K2 / A2. semantic structure → accessible interaction → reviewable AI-assisted change.

- Semantic document structure
- Accessible name computation
- Keyboard and focus contract
- Contrast and non-colour cues
- Validation error experience
- Lighthouse and axe evidence workflow
- Prompt constraints for accessible output
- Git diff and review guardrails

Activity 3: Build an Accessible Razor Pages Shell · Slides 93–95

Create semantic navigation, visible keyboard focus and a labelled customer-contact entry point. Evidence: an accessible layout and Contacts index page with a reproducible keyboard checklist.

Activity 4: Audit Accessibility with axe and Manual Evidence · Slides 96–98

Create a repeatable automated-plus-manual accessibility gate for the running Razor Pages UI. Evidence: axe JSON evidence, a manual keyboard checklist and a reviewed remediation diff.

Topic 3: Razor Pages Controls, Binding and Persistence · Slides 100–140

Alignment: LO3 · K3 / A3. page-focused UI → binding and validation → antiforgery → durable state.

- Razor Page execution model
- Razor Pages routing conventions
- Service boundary behind the PageModel
- Model binding input sources
- DataAnnotations and server validation
- Tag Helpers generate consistent controls
- Antiforgery request validation
- EF Core SQLite persistence

Activity 5: Create and List CRM Contacts with Razor Pages · Slides 134–136

Implement page-focused create/list handlers through an injectable contact store. Evidence: input model, IContactStore boundary, create/list PageModels and successful PRG navigation.

Activity 6: Enforce Validation and Antiforgery Controls · Slides 137–139

Reject invalid or forged commands while returning accessible recovery guidance. Evidence: bounded DataAnnotations, field/summary errors, antiforgery proof and negative-path tests.

Topic 4: Controller APIs and Component Interoperability · Slides 141–181

Alignment: LO4 · K4 / A4. resource contract → controller boundary → persistence → observable HTTP evidence.

- Resource-oriented HTTP semantics
- Controller routing and action selection
- DTO and domain boundary
- API model binding and automatic 400
- CRUD status and concurrency contract
- Async EF Core API path
- OpenAPI as interoperability contract
- Integration tests and ProblemDetails

Activity 7: Expose a Controller-Based CRUD API · Slides 175–177

Add a JSON controller surface that reuses the application service without exposing persistence entities. Evidence: controller routes, DTOs, 201/200/400/404 behaviour and curl evidence.

Activity 8: Prove Persistence and Interoperability · Slides 178–180

Replace volatile state with EF Core SQLite, publish an OpenAPI contract and verify components through integration tests. Evidence: SQLite migration, persistent CRUD path, openapi.json and HTTP integration tests.

Topic 5: Production Hosting, Azure and Design Evidence · Slides 182–222

Alignment: LO5 · K5 / A5. deployable artifact → observable service → safe release → traceable design record.

- Environment-specific configuration and secrets
- Structured logging and health checks
- Publish output and hosting boundary
- Azure App Service component model
- Deployment slots and safe promotion
- Managed identity and database access
- CI/CD gates and smoke evidence
- ADR, C4 and requirements traceability

Activity 9: Prepare an Azure-Ready Release · Slides 216–218

Create a production-neutral publish artifact with validated configuration, health endpoints and smoke evidence. Evidence: Release publish directory, health checks, deployment configuration matrix and smoke-test script.

Activity 10: Produce the Traceable Design and Release Record · Slides 219–221

Map user specifications to components, interface contracts, decisions and verification evidence. Evidence: C4-style diagrams, ADR, requirements traceability matrix, API contract hash and release checklist.

Resources Required

- Computer with .NET 10 SDK, Git, browser and code editor
- Optional Node.js for axe CLI
- Course activities/ tree with ten self-contained LocalCRM snapshots
- Fictional contacts.json and contacts.csv in every Activity
- LMS access for attendance and assessment

Assessment

- Written Assessment — Short-Answer Questions, 16:00–17:00 on Day 2, open book, K1–K5.
- Practical Performance — 17:00–18:00 on Day 2, open book, A1–A5.
- Minimum 75% SSG digital attendance and Competent outcome are funding conditions; eligibility and terms apply.
- Courseware, attendance and submissions: https://lms-tms.tertiaryinfotech.com/